

Understanding Model Parameters

From a single weight to autograd — how models learn

CONTENTS

1. What a Parameter Actually Is
2. Parameter vs. Hyperparameter
3. A Parameter You Can Watch — NumPy
4. Reading the Training Output
5. Where the Change Really Happens
6. The Same Idea in PyTorch
7. NumPy vs. PyTorch — Side by Side
8. Mental-Model Checklist

1 What a Parameter Actually Is

A **parameter** is a value the model learns from data. It is not something you set by hand — it starts at some initial value and gets adjusted during training until the model maps inputs to outputs well. **Weights** and **biases** are the classic examples.

Take the simplest possible model, a linear one:

$$y = w * x + b$$

Here **w** (the weight) and **b** (the bias) are the parameters. The model's entire “knowledge” lives in these two numbers. Training means finding the values of **w** and **b** that make predictions match reality.

2 Parameter vs. Hyperparameter

This distinction trips up nearly every beginner, so it is worth stating plainly:

	PARAMETER	HYPERPARAMETER
Examples	weight w , bias b	learning rate, epochs
Set by	learned from data	chosen by you
Updated by	gradient descent	never updated in training
Controls	the mapping itself	how learning happens

TIP

If a value gets nudged automatically every training step, it is a parameter. If you typed it in and it stays put, it is a hyperparameter.

3

A Parameter You Can Watch — NumPy

This trains a one-parameter-pair model in plain NumPy so you can see the parameters change on every step. The true relationship is $y = 2x + 1$; the model starts ignorant ($w=0$, $b=0$) and discovers those values from data alone.

```

import numpy as np

# --- Data: the "truth" is  $y = 2x + 1$  ---
X = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([3.0, 5.0, 7.0, 9.0])

# --- Parameters: learned values, initialized before training ---
w = 0.0 # weight
b = 0.0 # bias

# --- Hyperparameters: set by us, NOT learned ---
learning_rate = 0.01
epochs = 1000

n = len(X)

for epoch in range(epochs):
    # Forward pass: map inputs -> outputs
    y_pred = w * X + b

    # Loss: how wrong we are (mean squared error)
    error = y_pred - y
    loss = np.mean(error ** 2)

    # Gradients: how loss changes w.r.t. each parameter
    grad_w = (2 / n) * np.sum(error * X)
    grad_b = (2 / n) * np.sum(error)

    # Gradient descent: update the PARAMETERS
    w -= learning_rate * grad_w
    b -= learning_rate * grad_b

```

4

Reading the Training Output

Printing w , b , and the loss every 200 epochs gives this:

```
epoch 0 | loss=41.0000 | w=0.300 b=0.100
epoch 200 | loss=0.0273 | w=2.099 b=0.706
epoch 400 | loss=0.0087 | w=2.056 b=0.836
epoch 600 | loss=0.0028 | w=2.032 b=0.907
epoch 800 | loss=0.0009 | w=2.018 b=0.947

Learned parameters: w=1.990, b=1.030 (target: w=2, b=1)
Prediction for x=5: 10.981 (true: 11)
```

Read it like a dial being tuned toward a clear radio station (low loss):

- **Epoch 0:** w jumped from 0.0 to 0.300 in one step. The error was huge, so the gradient was big, so the nudge was big. Loss was 41 — very wrong.
- **Epoch 200:** $w=2.099$ already. The weight got close fast; loss fell from 41 to 0.03.
- **Later epochs:** the nudges shrink because the error is tiny. b keeps climbing slowly (0.706 $\rightarrow$ 0.836 $\rightarrow$ 0.907) because it converges more slowly than w here. Progress slows as you approach the target — like coasting into a parking spot rather than slamming the brakes.

5

Where the Change Really Happens

A natural beginner question: is NumPy secretly doing the learning? **No**. The parameter change happens in *your own two lines*:

```
w -= learning_rate * grad_w
b -= learning_rate * grad_b
```

NumPy is just a fast calculator. It multiplies and sums arrays, but it has no idea what “training” or “loss” or “parameter” means. You wrote the logic that adjusts w and b . Delete those two lines and the parameters sit at 0.0 forever, no matter how many epochs run.

Each epoch does three things: **(1) guess** with the current parameters, **(2) measure how wrong** the guess is (the loss), and **(3) nudge each parameter** the opposite way to the gradient. The gradient is just a signpost saying “loss gets worse this way”; the learning rate keeps the step small so you don’t overshoot. Repeat 1000 times and the parameters creep to the values that make the loss tiny. That is the whole trick.

WATCH OUT

There is no hidden magic in the library. “Training a model” is exactly that loop — guess, measure error, step in the better direction, repeat — stripped to its bones.

6

The Same Idea in PyTorch

Frameworks wrap that loop. `nn.Parameter` tags a tensor as trainable, and the optimizer performs the update step for you:

```
import torch
import torch.nn as nn

X = torch.tensor([[1.0], [2.0], [3.0], [4.0]])
y = torch.tensor([[3.0], [5.0], [7.0], [9.0]])

model = nn.Linear(1, 1) # creates weight + bias parameters
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
loss_fn = nn.MSELoss()

for epoch in range(1000):
    y_pred = model(X)
    loss = loss_fn(y_pred, y)
    optimizer.zero_grad()
    loss.backward() # computes gradients per parameter
    optimizer.step() # updates the parameters

for name, param in model.named_parameters():
    print(name, param.data)
```

`model.parameters()` is exactly the set of tensors the optimizer may touch. This is also where the definition's edge cases live: a **frozen** parameter has `requires_grad = False` (excluded from updates during fine-tuning), and **parameter-efficient adapters** like LoRA freeze the originals and train a small extra set instead.

7

NumPy vs. PyTorch — Side by Side

Both examples do the exact same thing — learn w and b for $y = 2x + 1$ via gradient descent. They differ only in who does the calculus:

STEP	NUMPY (BY HAND)	PYTORCH (AUTOMATED)
Forward pass	you write $y = w \cdot X + b$	<code>model(X)</code>

STEP	NUMPY (BY HAND)	PYTORCH (AUTOMATED)
Gradients	you derive the formulas	loss.backward() (autograd)
Update	you write <code>w -= lr*grad</code>	optimizer.step()
Best for	seeing the mechanics	scaling to millions of params

TIP

NumPy is transparent but you must re-derive gradients for every new model — infeasible for anything deep. PyTorch scales because autograd derives gradients automatically. That is why real models use it.

8 Mental-Model Checklist

1

Parameters are learned, not typed

`w` and `b` start at an initial value and are updated from data.

2

Hyperparameters configure the process

Learning rate and epochs are chosen by you and never updated in training.

3

The forward pass maps inputs to outputs

$y = w \cdot x + b$ uses the current parameter values to make a prediction.

4

Loss measures how wrong the guess is

One number; lower is better. It is the signal that drives learning.

5

Gradients point the way downhill

They say which direction increases loss; you step the opposite way.

6

The update line does the learning

`w -= lr*grad_w` is where the parameter actually changes — in your code, not the library.

7

Frameworks automate the same loop

`backward()` and `step()` replace your hand-written gradient and update lines.